XR Study Whiteboard Code Study Guide

XR Study Whiteboard

Andhika Nayaka Arya Wibowo

The University of Queensland

DECO2300: Digital Prototyping and Extended Reality

DECO2300 Teaching Team

21 October 2026

XR Study Whiteboard Code Study Guide

How to Use This Guide

This guide explains the code already present in the project. It does not add gameplay features or change the tested APK. Start with the feature map, then read the scripts in the suggested order. The guide covers all 34 custom C# files. A separate working index retains method line numbers, source hashes, and GitHub links. There are 27 runtime scripts and seven Unity Editor tools.

Be ready to show the source rather than only describing the APK. An APK is the compiled Android application. The readable C# scripts, Unity scene, package manifest, and project settings are the source material. The assessor may ask you to open a relevant script or explain how a controller action produces a result.

The Two Main Folders

Folder	What it contains and why it is separate
Whiteboard	The real Unity project. Unity Hub opens this folder. It contains Assets, Packages, and ProjectSettings. Its location was preserved so existing scene and asset references keep working.
Interactive Prototype 2	The assessment package. It contains the plan, journal, forms, evidence, evaluation, video, APK, and a portable source archive. It organises submission material without creating another gameplay implementation.
Interactive Prototype 2/Source Code	A portable Unity project ZIP. The authoritative editable project is still Whiteboard. The archive contains tracked Assets, Packages, and ProjectSettings.
Working Files/Interactive Prototype 2/Tools	Python tools that build or check documents. They do not implement the VR classroom and do not run inside the Quest APK.

The separate assessment folder is an organisational decision. It helps a marker find the required deliverables. It is not an extra Unity project, a different XR system, or a replacement for the C# code.

Unity Project Folder Map

Location inside Whiteboard	Purpose and reason
Assets/XRStudyWhiteboard/Scripts/Core	Overall classroom setup and shared whiteboard state. These scripts connect systems rather than painting individual pixels or handling one button.
Assets/XRStudyWhiteboard/Scripts/Whiteboard	Whiteboard and paper writing surfaces, strokes, and paper tool selection. Keeping drawing here separates visual ink from device input.

Location inside Whiteboard	Purpose and reason
Assets/ XRStudyWhiteboard/ Scripts/XR	Controller input, movement, camera destinations, simulator input ownership, and haptics. Device behaviour can change without rewriting the brush.
Assets/ XRStudyWhiteboard/ Scripts/UI	Buttons, world-space menus, status, guide, layout, and icons. These scripts ask the state or drawing system to act. They do not each keep a separate copy of the board's colour state.
Assets/ XRStudyWhiteboard/ Scripts/Recording	In-memory board action records and timed playback. This is different from recording Syifa with a camera.
Assets/ XRStudyWhiteboard/ Scripts/Hands	Secondary hand-pinch input and a future gesture extension point. Physical controllers are the evaluated primary interaction.
Assets/ XRStudyWhiteboard/ Editor	Scene construction, validation, and editor convenience tools. Unity treats this as an Editor-only folder, so its UnityEditor code is excluded from the normal player build.
Assets/ XRStudyWhiteboard/ Scenes	XRStudyClassroom.unity stores the scene objects and serialized component references. A scene is data connecting components, not another C# controller script.
Assets/ XRStudyWhiteboard/ Art/Classroom	The imported classroom FBX and textures. Visual environment content is separate from the project's interactive board and paper components. Do not describe imported art as a mesh model you coded from scratch.
Assets/ XRStudyWhiteboard/ Materials	Material assets that control surface appearance. Runtime canvases create owned material instances for their changing textures.
Assets/ XRStudyWhiteboard/ Prefabs, Input, and Audio	Organisational folders exist, but that does not mean every planned asset is implemented here. The active XR rig and action asset are reused from the template and Samples paths below. Audio recording is not implemented.
Assets/ VRTemplateAssets and Assets/Samples	Existing Unity template and sample assets. The project reuses the XR Origin, controller setup, hand support, and XRI actions instead of recreating the XR framework.
Packages/manifest.json and packages-lock.json	Declare and lock Unity package dependencies. They are needed when another machine imports the source project.
ProjectSettings	Unity version, Android/XR configuration, layers, build scenes, and other project configuration. Source submission needs these settings as well as C# files.
Library, Temp, Logs, and build backup folders	Generated or local data. They are excluded from the source archive and Git rather than submitted as the source implementation.

Unity .meta files hold asset identifiers called GUIDs. Keep each asset with its .meta file. Deleting those files can break scene references even when the C# text is still present. The XRStudyWhiteboard namespace groups the project's classes under one name and helps avoid naming conflicts with packages.

Feature Map: Where to Look First

Feature	Main code and connection
Right-controller writing	XRWhiteboardInteractor.LateUpdate reads its configured Select drawing action and ray, chooses UI, paper, or board, and sends the hit to the appropriate drawing path. See the binding distinction below.
Whiteboard ink	WhiteboardDrawer.DrawAtUV controls stroke start and continuation. WhiteboardCanvas stores and paints the pixel texture.
Table paper ink	PaperNoteCanvas.DrawAtUV paints that paper's separate texture. PaperTool holds the selected paper mode.

Feature	Main code and connection
Marker colour	WhiteboardColourButton.SelectColour calls XRStudyWhiteboardManager.SetColour. WhiteboardCanvas uses GetCurrentInkColour when painting.
Marker versus eraser	WhiteboardToolButton.SelectTool changes the manager's CurrentTool. The canvas selects marker ink or a larger white brush.
Clear Board	WhiteboardClearButton requests a confirmation. ClearBoardConfirmation calls the manager's confirm or cancel method.
Pencil, Eraser, Clear Paper	StudyTableToolMenu selects PaperTool or calls ClearNote on its own PaperNoteCanvas.
Paper cannot be grabbed	ClassroomAssetRuntimeSetup.ConfigurePaperForWriting removes the paper's XRGrabInteractable and Rigidbody.
Left X to the board	XRStudyRoomLocomotion.HandleQuestNavigationButtons calls NavigateToWhiteboard.
Left Y through tables	HandleQuestNavigationButtons calls NavigateToNextTable. StudyTableTeleportPoint and SeatAnchor supply destinations.
Walking and turning	HandleXRInput reads the left stick for movement and the right stick for snap turn. MoveInViewDirection and Turn move the rig.
Guide and right A	VRControllerGuide.Build creates the guide. Update reads right primaryButton and toggles visibility.
Record, Stop, Play	WhiteboardControlPanel calls WhiteboardSessionRecorder. WhiteboardRecordingSession holds the event data.
Keyboard T/Y	XRStudyRoomLocomotion.SelectDesktopController and DesktopSimulatorInputOwnership choose a simulated controller. These keyboard keys are different from the physical Quest Y button.
Floor teleport ray	The scene builder adds an XRI TeleportationArea and reuses the template's teleport infrastructure. The X/Y destination shortcuts are a separate custom navigation route.

Core Scripts

All script paths in the next tables are relative to Assets/XRStudyWhiteboard. Runtime means the code can be used while the application is running. It does not mean every optional path was evaluated by participants.

Script	Purpose, main methods, and why it is separate
Scripts/Core/ XRStudyWhiteboardManager.cs	Holds CurrentTool and CurrentColour in one place. SetTool at line 54 and SetColour at line 64 end the previous stroke before changing state. RequestClear at line 95, ConfirmClear at line 108, and CancelClear coordinate clearing. EnsureRecorder creates/configures the recorder during play. StateChanged tells displays and recording listeners to refresh. A shared manager prevents the button label and brush from using conflicting colour or tool values.
Scripts/Core/ ClassroomAssetRuntimeSetup.cs	Adapts the imported classroom to the interactive systems at startup. Start at line 31 fits the room, places the functional board, and creates student stationery. FindTabletopBounds locates usable desk surfaces. CreateStationerySet at line 457 attaches PaperNoteCanvas, StudyTableToolMenu, and StudyTableTeleportPoint. ConfigurePaperForWriting at line 486 removes paper grabbing. This keeps imported visual geometry separate from the project-owned interactions.

Drawing and Paper Scripts

Script	Purpose, main methods, and why it is separate
Scripts/Whiteboard/ WhiteboardCanvas.cs	Owns the board's Texture2D, Color32 pixel array, runtime material, brush sizes, and stroke filtering. InitializeSurface at line 99 creates the surface. TryGetUV at line 143 converts a valid front-face hit into texture coordinates. BeginStroke at line 200, ContinueStroke at line 218, and EndStroke at line 284 define a stroke. Stamp and StampSegment paint pixels. ClearBoard fills white. DrawingEvent lets the recorder observe changes without putting recording logic into every input script.
Scripts/Whiteboard/ WhiteboardDrawer.cs	A small bridge between input and the board canvas. DrawAtUV at line 26 begins a new stroke or continues an open one. EndStroke at line 53 ends it. It blocks live drawing while playback or clear confirmation is active. Controllers and the optional hand path can call the same entry point instead of duplicating stroke rules.
Scripts/Whiteboard/ PaperNoteCanvas.cs	Owns an individual paper texture and a fine dark pencil brush. DrawAtUV at line 150 draws or erases. TryGetNearest at line 298 chooses the closest valid paper. TryGetPaperIntersection at line 328 uses the paper's top plane. ClearNote at line 289 clears only this instance. Each paper has its own pixels, so clearing one desk does not erase every desk or the whiteboard.
Scripts/Whiteboard/ PaperTool.cs	Defines PaperToolKind and keeps the shared paper tool selection. Select at line 48 changes Pencil or Eraser and raises SelectionChanged. Compatibility handlers can observe held tools, but current generated paper is fixed and tools are selected through the menu. Selected paper tool is shared across desks while each paper's ink remains separate. This prevents paper tool state from overwriting the main whiteboard's marker/eraser state.

Controller, Navigation, and Simulator Scripts

Script	Purpose, main methods, and why it is separate
Scripts/XR/ XRWhiteboardInteractor .cs	The main physical-controller input router. ResolveControllerTriggerAction at line 113 reads the XRI Select action, with a device-trigger fallback in TryGetControllerInput at line 866. LateUpdate at line 158 checks UI before paper and board drawing. RestrictToolUiToRightController at line 77 disables left-controller tool UI participation without removing its locomotion. Keeping input here lets the canvas receive device-independent UV points. The file also contains desktop pointer compatibility, so it is larger than the brush bridge.
Scripts/XR/ XRStudyRoomLocomotion.cs	Handles movement, snap turning, fixed destinations, and desktop navigation. HandleXRInput at line 515 reads physical sticks. HandleQuestNavigationButtons at line 543 detects new X/Y presses. NavigateToWhiteboard at line 416 calculates a suitable board-facing destination. NavigateToNextTable at line 559 cycles table indices. AlignCameraToPose at line 480 moves the whole XR Origin around the actual tracked head. This keeps camera and controllers under one moving parent.
Scripts/XR/ SeatAnchor.cs	Represents an intended camera/head position and a horizontal facing direction. FindOrCreate at line 14 stores anchors under NavigationAnchors. It provides destination data rather than moving the player itself. Separating the target from the movement code lets several tables use the same navigation algorithm.
Scripts/XR/ StudyTableTeleportPoint.cs	Connects one paper and desk to its SeatAnchor. Initialize at line 32 derives eye position from desk bounds and registers a table index. OnEnable and OnDisable keep the active table list current. It knows where the desk is, while locomotion decides when and how to move the rig there.
Scripts/XR/ DesktopSimulatorInput Ownership.cs	Restricts which keyboard and mouse actions the simulator owns. SetExplicitControllerMode at line 60 changes controller posing mode. GatedMouse and Dispose create gated actions and restore the previous references. This prevents ordinary room movement from also moving simulated hands and makes the faster-controller symptom easier to isolate. It supports editor testing and is not the Quest hardware tracking implementation.

Script	Purpose, main methods, and why it is separate
Scripts/XR/ ControllerHaptics.cs	PulseRightController at line 7 requests a small right-controller haptic impulse. Shared feedback code avoids repeating device lookup and pulse settings across colour, tool, and recording buttons. A haptic request in code is not proof that every device produced feedback during testing.

User Interface Scripts

Script	Purpose, main methods, and why it is separate
Scripts/UI/ WhiteboardColourButton.cs	Binds one button to one WhiteboardColour. SelectColour at line 42 asks the manager to change it. Refresh at line 48 updates the swatch and selection outline. The button chooses a value rather than painting the board itself.
Scripts/UI/ WhiteboardToolButton.cs	Binds Marker or Eraser to the manager. SelectTool at line 40 requests the change and Refresh updates its active outline. It shares the same state source as colour controls and the canvas.
Scripts/UI/ WhiteboardClearButton.cs	RequestClear at line 24 asks the manager to open confirmation. It deliberately does not directly wipe the canvas, so an accidental first click can be cancelled.
Scripts/UI/ ClearBoardConfirmation.cs	Show, Hide, Confirm, and Cancel control the clear overlay. BindListeners at line 29 avoids binding the same button twice. The confirmation layout is separate from the board's pixel-clearing operation.
Scripts/UI/ WhiteboardStatusDisplay.cs	Refresh at line 30 shows the active tool and colour. InitializeCompact supports the later compact panel. SetDrawingState changes activity feedback. Display code observes state and should not become a second state manager.
Scripts/UI/ WhiteboardControlPanel.cs	Initialize at line 22 rearranges and styles the board controls and adds recording controls. RecordOrStop at line 123 calls the recorder. RefreshSession at line 138 updates labels, timing, and button availability. RefreshSelection marks active choices. During playback it disables drawing controls so the replay is not changed by live selection. The actual replay algorithm remains in the recorder.
Scripts/UI/ StudyTableToolMenu.cs	Builds the menu beside a paper. BuildMenu at line 118 creates Pencil, Eraser, and Clear Paper. LateUpdate at line 156 shows the closest occupied desk menu instead of cluttering every desk. SelectTool at line 248 updates PaperTool. ClearPaper at line 254 clears its linked note. Pointer handling avoids repeated clicks and consumes menu hits before they become ink.
Scripts/UI/ VRControllerGuide.cs	Build at line 36 creates a view-relative world-space guide explaining the two controller roles. Update at line 103 toggles it on a new right-A press. ShowGuide at line 123 changes visibility. The default initial display is 14 seconds and it can be reopened. Guidance is separated from movement so changing instructions does not change navigation behaviour.
Scripts/UI/ XRUIInputSetup.cs	Awake at line 17 connects XRI UI actions to XRUIInputModule. Initialize supplies the action asset and module. XR input is enabled, and automatic mouse input is disabled because the custom desktop bridge already handles that pointer. This avoids delivering one desktop click twice.
Scripts/UI/ StudyUiStyle.cs	Shared creation and styling helpers for panels, buttons, and text. CreateButton at line 60 and StyleButton at line 71 provide consistent active/hover appearance. RoundedSprite produces the common shape. It keeps layout/appearance details out of each feature's state logic.
Scripts/UI/ StudySessionIcon.cs	Draws Record, Stop, and Play shapes through OnPopulateMesh at line 19. SetKind changes the symbol. Using a small mesh avoids depending on an external icon font and keeps icon generation independent of recording state.

Recording and Hand Scripts

Script	Purpose, main methods, and why it is separate
Scripts/Recording/ WhiteboardRecordingSession.cs	Defines Idle, Recording, and Playing states plus StrokeBegin, StrokePoint, StrokeEnd, ToolChange, ColourChange, and Clear events. Each event stores timing, position, stroke ID, tool, and colour. The session keeps the starting pixel snapshot and event list. This is a data model rather than a component that reads controller input.

Script	Purpose, main methods, and why it is separate
Scripts/Recording/ WhiteboardSessionRecorder.cs	Configure at line 35 subscribes to board and manager events. StartRecording at line 48 captures starting pixels and starts the clock. Stop at line 73 closes the last stroke. Play at line 97 restores the start and ApplyEventsThrough at line 125 replays events at their recorded times. Separating data, recorder, UI, and canvas makes playback testable without recording camera frames. The session is memory-only and currently covers the whiteboard, not paper or microphone audio.
Scripts/Hands/ HandWhiteboardInteractor.cs	Reads right index/thumb joint poses and uses separate start/release pinch distances. Update at line 36 sends successful board hits to the same WhiteboardDrawer. It ends strokes when tracking or contact is lost and skips the editor to prevent a second input writer. This is a secondary path, not evidence that physical hand tracking was tested in the controller study.
Scripts/Hands/ HandToolSwitchGesture.cs	Contains an enabledForFutureIteration flag and a property exposing it. It is an extension point for the original two-finger swipe idea. There is no implemented swipe detector in this file. Tool buttons are the functional mechanism. Keeping a future placeholder is not the same as claiming a finished gesture feature.

Editor Tools

These seven scripts live directly in Assets/XRStudyWhiteboard/Editor. They help construct and check the project in Unity Editor. They are not the scripts the headset executes for drawing.

Script	Purpose, main methods, and why it is separate
XRStudyWhiteboardSceneBuilder.cs	BuildClassroom at line 45 creates the classroom scene, functional board, UI, XR rig references, floor teleport area, materials, and build-scene entry. It reuses the template XR Origin and XRI actions and places the imported FBX. OpenMainScene opens the scene. A repeatable builder makes setup easier to repair. It creates/replaces scene content, so do not run it casually just before assessment on a tuned scene.
XRStudyWhiteboardValidation.cs	ValidateClassroom at line 41 checks scene/configuration and missing references. BuildAndroidPlayerValidation at line 88 builds a validation player. Drawing and menu smoke tests exercise important behaviours. AddDeviceSimulatorForEditorTest and RemoveDeviceSimulatorForEditorTest manage editor-only testing setup. These checks catch structural mistakes but do not replace a headset user test.
XRStudyWhiteboardSessionValidation.cs	ValidateDrawingAndRecording at line 12 tests surface coordinates, stroke boundaries, and session replay. ValidateSession compares replay pixels and events. Dedicated recording tests can check timing and replay behaviour independently of the UI's appearance.
XRStudyNavigationValidation.cs	Validates Quest interaction contracts, camera-anchor alignment, simulator input ownership, and T/Y selection. Aligning the head with translated/scaled parents is tested separately from artwork. These tests explain how earlier movement problems were checked rather than merely hidden.
XRStudyUiValidation.cs	ValidateStudyControls at line 13 checks UI behaviour, session icon geometry, and table pointer handling. It focuses on clicks and feedback, including avoiding repeated activation, rather than deciding which colour the manager should use.
XRStudyWhiteboardEditorStartup.cs	OpenMainSceneWhenUntitled at line 18 opens the classroom when the editor starts with an untitled scene. It is a convenience tool. It does not generate participant results or run in the Android application.
XRStudyWhiteboardPlayFocus.cs	OnPlayModeStateChanged at line 17 and FocusGameView at line 25 focus the Game view after entering Play Mode. This helps keyboard testing reach the intended window. It does not control Quest headset focus or tracking.

What Happens When You Write on the Whiteboard

Step	What the code does and why
1. Read a device	OpenXR/XRI supplies a controller pose and input action. XRWhiteboardInteractor resolves the configured right Select action and its ray origin. Tracking itself comes from the XR framework.
2. Give UI priority	The interactor checks controls before drawing. A colour or tool click should be consumed as UI instead of leaving an accidental mark behind the panel.
3. Find the surface	WhiteboardCanvas.TryGetUV intersects the visible front plane for a BoxCollider and checks bounds. It rejects side and back hits that could create spikes.
4. Convert to UV	A world-space contact becomes a 0-to-1 point across the board texture. The board supports horizontal flipping to match the actual visible cube-face texture orientation.
5. Start or continue	WhiteboardDrawer decides whether the canvas should BeginStroke or ContinueStroke. Releasing the drawing action or losing the hit ends the stroke.
6. Paint a continuous segment	The canvas filters input and interpolates segments between samples. StampSegment paints a capsule-like brush region so fast movement does not become disconnected dots. Large untrusted jumps are rejected instead of joined across empty space.
7. Upload changed pixels	The canvas marks the texture dirty and applies changed pixels during its update cycle. The material shows the runtime texture on the board.

The written line is texture-based ink. It is not a handwriting-recognition result, a saved text document, or a separate 3D mesh for every letter. This approach is simple for a bounded prototype but uses CPU pixel work and has not been demonstrated as a fully profiled production renderer.

What Changes for Paper

Paper lies horizontally, so PaperNoteCanvas uses local X/Z coordinates on the top plane instead of the board's local X/Y front plane. The paper code measures the nearest valid top-plane hit, not an arbitrary side-face collision. The visible writing surface is oriented to agree with those coordinates. This matters because a mismatch between contact mapping and display texture can make ink appear mirrored or move in the wrong direction.

Each PaperNoteCanvas owns its own pixel array and texture. PaperTool selection is shared, so choosing Eraser can carry to another desk, but the actual notes are not shared. The paper pencil is dark and does not automatically adopt the whiteboard's red, blue, or green marker selection. The wider eraser paints white over its own surface. Clear Paper calls ClearNote on the menu's linked paper. Clear Board goes through the manager and confirmation instead.

The paper is ungrabbable because ConfigurePaperForWriting removes the grab component and Rigidbody. A collider remains useful for locating the writing plane. Removing grab behaviour is different from removing the collider or disabling the whole paper object.

Why Teleporting Moves the Whole Rig

The camera and controllers are children of the XR Origin. Moving only the camera could leave hands behind. Moving only the origin to a fixed location can also place the user's actual head incorrectly when the head has a room-scale offset.

AlignCameraToPose first calculates the horizontal yaw difference between the actual camera direction and destination direction. It rotates the origin around the camera, then translates the origin by target camera position minus current camera position. It preserves tracked local pose, pitch, roll, and rig scale. This is why the anchor describes eye position rather than a chair's floor position.

The board destination is calculated from the board bounds, approximately 1.75 metres in front, with eye height clamped between 1.55 and 1.75 metres. Table anchors use desk bounds and eye height above the desktop. Those are current implementation values, not universal comfort guarantees. Desktop view pitch and field of view are adjusted for testing, while headset pitch remains under the user's tracked head control.

The physical X and Y buttons use rising-edge detection. A held button is not treated as a new press every frame. The next-table index wraps using modulo so Y cycles through the registered tables.

Colour, Erasing, and Events

The manager owns the colour enum rather than asking each button to keep its own global state. A colour button sends SetColour. The manager ends the old stroke, changes CurrentColour, raises StateChanged, and requests feedback. Displays refresh their selected

appearance, and the recorder can log the change. This is an event-driven relationship: listeners respond when state changes rather than every component independently inventing its own value.

Eraser mode uses the larger brush and white paint on the canvas. It does not remove the board GameObject. Clear Board replaces the whole pixel buffer with white after confirmation. Clear Paper does the corresponding operation on one paper. A tool change ends an open stroke so one continuous gesture does not unexpectedly switch colour or width halfway through its previous segment.

The Two Different Meanings of Recording

Recording type	What it actually stores
The supplied Syifa MOV	Camera video and audio of the physical testing session, including headset/controller use and the running classroom on the laptop. It is assessment evidence stored in Media. It was recorded outside the in-app drawing recorder.
In-app Record, Stop, Play	A starting whiteboard pixel snapshot and timed drawing/tool/colour/clear events in memory. Play restores the start and reuses the canvas brush in event order. It is not a camera video, does not record microphone audio, and does not persist after restart.

WhiteboardSessionRecorder uses separate Idle, Recording, and Playing states. It records drawing events only while Recording. During playback the canvas and manager apply recorded events through a dedicated path, so playback is not recorded back into the event list. This prevents a replay from continually growing its own recording. Live board input and drawing controls are blocked during playback to keep the result consistent.

Select and Activate Are Action Names

The intended control guide describes right-trigger writing, but an action name alone does not establish its physical binding. The saved scene references the XRI Default Input Actions asset. In that asset, the controller Select binding uses GripButton, while Activate uses TriggerButton. XRWhiteboardInteractor.ResolveControllerTriggerAction currently requests XRI Right Interaction/Select. Its direct device trigger fallback is reached only if no action was resolved. The code comment describing Select as the index trigger does not match the saved bindings.

Source item	What is actually present
XRI Right Interaction/Select	Bound to the right controller GripButton, alongside alternative hand/simple-controller bindings.
XRI Right Interaction/Activate	Bound to the right controller TriggerButton.
XRWhiteboardInteractor	Requests Select and calls IsPressed when that action is found. The physical trigger fallback is a separate branch.
Existing tested APK	The student reports successful operation. Its exact build-time source and bindings were not extracted from the APK during this review.

For a code explanation, show both the action lookup and its actual binding rather than treating Select and trigger as synonyms. A runtime override or different build-time asset would need evidence. No runtime code, bindings, or APK were changed for this study guide. Confirm the mapping before making a future build rather than assuming the explanatory comment is authoritative.

Unity Framework Versus Project Code

Existing framework or asset	What the project uses it for
Unity and URP	Scene objects, transforms, physics, textures, rendering, materials, and the game loop. The custom scripts do not implement a game engine.
OpenXR and XR Management	Starting and connecting the XR runtime. Having the package installed alone is not proof that Android XR startup is configured correctly.
XR Interaction Toolkit	XR Origin, controller rays, UI interaction, teleportation infrastructure, and interactable behaviour. The scene builder reuses existing rig assets.
Input System and XRI Default Input Actions	Controller button and UI action bindings. The active action asset is under Assets/Samples/XR Interaction Toolkit/3.4.1/Starter Assets.
XR Hands	Optional tracked hand-joint data for the secondary pinch path. It was not the input used in the physical-controller participant sessions.
TextMeshPro and Unity UI	World-space text, canvases, buttons, outlines, and event handling. StudyUiStyle gives these controls a consistent project-specific appearance.
Imported classroom FBX	Visual classroom geometry. ClassroomAssetRuntimeSetup fits it and places the project's functional interaction surfaces around it.

A useful explanation is: the project reuses established XR infrastructure and adds the classroom interaction rules, drawing surfaces, navigation destinations, menus, and timed action replay. Do not claim that the hardware tracking, Unity packages, or imported art were written from scratch.

Questions to Practise

Possible question	An explanation grounded in this project
Why did you create separate folders?	Runtime input, drawing, state, UI, recording, and editor tools have different jobs. Separating them makes it easier to find a fault and change one feature without rewriting all other features. The assessment folder then groups deliverables without moving the live Unity project.
Where does a red stroke come from?	The red button calls the manager's SetColour. The interactor supplies valid UV points through the drawer. The canvas reads the manager's selected colour and paints the texture. The button itself does not paint pixels.

Possible question	An explanation grounded in this project
Why use a drawer and a canvas?	The drawer coordinates start/continue/end requests. The canvas owns texture geometry and painting. Controllers and hands can share the drawer without duplicating brush code.
Why have a separate paper canvas?	Paper uses a horizontal plane, a fine pencil, per-desk notes, and local clearing. It must not share a mutable texture with the main board or every other desk.
Why is paper fixed?	The note is a writing surface. Grabbing competed with the same trigger used for writing. Removing paper grab behaviour makes the task predictable while preserving its collider for hit mapping.
How did you address unwanted long lines?	End strokes on release or surface loss, match UV mapping to the displayed surface, reject large untrusted input jumps, and interpolate only valid nearby samples. Do not explain interpolation as accepting every jump indiscriminately.
Why were controllers moving faster in desktop testing?	The simulator and room movement could both consume keyboard input. DesktopSimulatorInputOwnership gives those inputs one owner and separates room movement from deliberate hand posing. Physical tracking still belongs to the XR runtime.
How do X and Y work?	The left primary and secondary buttons trigger board navigation and the next table on new presses. Table indices and camera anchors supply destinations. The right controller remains available for selecting and writing.
What does the recording feature save?	It keeps a starting board image and timestamped actions in memory. Playback restores the start and replays those actions. It is not an exported video and does not save a lesson to disk.
Can I show the APK instead of source?	The APK proves the compiled app can run on Android. The marker also needs the Unity source project or repository, including assets, package dependencies, settings, and scripts. The portable source ZIP is available locally.
Have you implemented every idea from the paper prototype?	No. Controller drawing is the evaluated route. Pinch is a secondary implementation, and the two-finger swipe file is an extension point. AR, multiplayer, audio recording, and saved notes are not completed features.
Does successful testing prove better learning?	It supports usability of the observed classroom tasks. It does not establish improved focus or learning without a separate educational study. Report high satisfaction alongside prompts and retries visible in the evidence.

Suggested Reading Order Before the Studio

Read XRStudyWhiteboardManager first, then the small WhiteboardDrawer. Next read WhiteboardCanvas.TryGetUV and its stroke methods, followed by XRWhiteboardInteractor.LateUpdate. Then compare PaperNoteCanvas and StudyTableToolMenu. Read HandleQuestNavigationButtons and AlignCameraToPose in locomotion, followed by WhiteboardSessionRecorder. Finally, use the Editor validation tables to understand what the checks cover and what still requires a real device.

Open XRStudyClassroom.unity in Unity and select the board, XR Origin, UI, and generated desk objects while studying their components. Some desk surfaces and menus are created at runtime, so compare Edit Mode and Play Mode. Do not save accidental Play Mode changes or rebuild the classroom just to inspect it.

Use the guide alongside the actual source. Practise following one controller action through its methods, then show the corresponding result in the classroom. Distinguish working features from future extension points when explaining the prototype.